UFSC CTS	**Universidade Federal de Santa Catarina** **Campus Araranguá - ARA** **Centro de Ciências, Tecnologias e Saúde** **Departamento de Computação** **PLANO DE ENSINO**

I. IDENTIFICAÇÃO DA DISCIPLINA			
CÓDIGO	**NOME DA DISCIPLINA**	**Nº DE HORAS-AULA SEMANAIS - TEÓRICAS**	**Nº DE HORAS-AULA SEMANAIS - PRÁTICAS**
DEC7532	Linguagem de Programação II	0	4
TOTAL DE HORAS - AULA SEMESTRAIS	**HORÁRIO TURMAS TEÓRICAS**	**HORÁRIO TURMAS PRÁTICAS**	**MODALIDADE**
72	02655A - 2.1620-2	02655A - 4.1620-2	Presencial

ATIVIDADE PRÁTICA T1 – Trabalho em Duplas

std::string, std::vector, std::pair, std::tuple, std::ifstream, std::ofstream, classes, encapsulmento, construtores, métodos de acesso

O exercício desenvolvido na aula do dia 19/10/2026 será prosseguido como trabalho T1 em duplas. Foi implementada a classe **Player** abaixo (*player.hpp*).

```
1   #include <iostream>
2
3   using namespace std;
4
5   class Player
6   {
7   private:
8       string m_name;
9       float m_life, m_attack, m_defense;
10
11  public:
12      Player();
13      Player(string name, float life=20, float att=3, float def=1);
14
15      string GetName() { return m_name; };
16      float GetLife();
17      float GetAttack();
18      float GetDefense();
19      void SetLife(float life);
20
21      bool Attack(Player &alvo);
22  };
```

O **Trabalho T1** consiste em utilizar o trabalho anterior e incrementar o escopo para uma batalha entre 2 times distintos à sua escolha. Exemplo: Humanos X ORCS, Vikings X Troianos, etc.

Devem ser utilizados no **main.cpp** 2 vetores, um para cada time, da seguinte maneira:

string filename_time1, filename_time2
cout « "Entre com o nome dos times 1:"
cin » filename_time1; //exemplo: humanos.txt – arquivos sem espaço no nome
cout « "Entre com o nome dos times 1:"
cin » filename_time2; //exemplo: orcs.txt – arquivos sem espaço no nome
vector<Player> Time1 = LoadTeam(filename_time1);
vector<Player> Time2 = LoadTeam(filename_time2);

Time 1 duelará com Time 2 até que todos os integrantes de um dos times sejam eliminados. O Time vencedor é o aquele onde ao menos 1 integrante sobreviver.

Pede-se:

1. Implementar o exposto acima;

2. No diretório do trabalho, criar uma pasta chamada "times", e nesta pasta deve e**xistir 5 arquivos**, correspondendo a 5 times diferentes **à sua escolha**. O arquivo terá a seguinte configuração, iniciando por um valor inteiro que corresponde a quantidade de guerreiros existentes (**não mais que 500**), seguido por seus nomes, como no exemplos abaixo:

humanos.txt

```
10
Rei Arthur (H)
Robin Hood
Beowulf
Siegfried
Rolando
El Cid
Ragnar Lodbrok
Mulan
Guilherme Tell
Bradamante
```

orcs.txt

```
14
Grommash Grito de Ferro
Throgar Quebra-Montanhas
Morgath Presa Negra
Durok Martelo Sangrento
Grishnak Devorador de Reis (H)
Azgrom Punho de Cinzas
Kharzug Rasga-Ceus
Gorvash Cranio de Aco (H)
Urzak Senhor das Cicatrizes
Drakthul Lamina Sombria
Zogmar Esmaga-Tronos
Brugash Rugido da Guerra
Skarnak Flagelo dos Homens
Vorgul Coracao de Furia
```

Após carregar os arquivos (*ifstream/ofstream*), atribuir aos mesmos **randomicamente** um valor de *life*, *attack*, *defense* (como no exercício em sala), e para aqueles marcados com "*(H)*", um poder diferenciado pois são heróis.

3. Implementar no arquivo *header* 2 novas classes: uma para um sistema de armas e outra para escudos, com atributos e poderes especiais, à sua escolha. Estas armas e escudos devem ser distribuídos de alguma forma aos lutadores. **Armas e escudos devem ser atributos da classe Player**.

4. Implementar para os lutadores **heróis** alguns **ataques-críticos** aleatórios, que serão aplicados a vários oponentes ao mesmo tempo no turno do ataque. Isso deve estar explícito no **main.cpp** através de um vetor de índices de oponentes que serão afetados. Deve existir um comentário no main para fácil identificação ao início da instrução contendo: ***//critical-hit starts here***

Entregável:

- Arquivo compactado no formato ZIP, contendo **main.cpp**; **player.hpp** (opcionalmente implementações em **player.cpp)**;
- Armas e escudos devem estar no arquivo **player.hpp** (opcionalmente as implementações em **player.cpp),** e devem ser atributos da classe Player.
- Colocar o nome dos integrantes como comentário logo no início do **main.cpp**;
- Gravar vídeo demonstrativo da implementação do jogo, explicando o código e demonstrando a execução. Ambos os alunos da dupla devem apresentar.
- Caso não consiga enviar o vídeo via Moodle, colocar o link de drive ou youtube privado no **main.cpp**, logo após o nome da dupla.